FoodDeliveryApp

Cart Management Module

Team 6

Members-

PES1UG23AM903-Lakshanyaa Shree

PES1UG23AM258-Samanvitha Periketi

PES1UG23AM256-Sai Prateek Sudhir

PES1UG23AM244-Ronit Das

Package: edu.classproject.cartrestaurant

1. Project Overview

FoodDeliveryApp is a Java-based application that simulates a food delivery platform where customers can browse restaurants, select menu items, manage their cart, and place orders. The overall system is divided into multiple independent modules that communicate through well-defined Java interfaces.

We Team 6 are responsible for the **Cart Management module**. This module allows users to create a cart for a selected restaurant, add items to the cart, update item quantities, view the cart contents, and clear the cart when needed. It also enforces important constraints such as ensuring that a single cart contains items from only one restaurant and validating item quantities to maintain consistency within the application.

Technology	Details
Language	Java
Build Tool	Maven
Testing Framework	JUnit 5
Data Storage	In-memory (HashMap — no database)
CI/CD	GitHub Actions (runs mvn test on every PR)
Architecture	Interface-driven, framework-free OOP

2. Synopsis

The Cart Management module is an important part of the Food Delivery Application and is responsible for handling all cart-related operations for users. In this project, our team worked on implementing the cart functionality that allows customers to create a cart for a selected restaurant, add menu items to the cart, update the quantity of items, view the items currently present in the cart, and clear the cart when required. The cart acts as a temporary space where users can collect the food items they wish to order before proceeding to checkout.

While developing this module, we ensured that certain rules are followed to maintain consistency in the system. One important rule is that a cart should contain items from only one restaurant at a time. This prevents issues during order processing. The system also checks for valid inputs, such as making sure that the quantity of an item is not negative and that required

values like customer ID and restaurant ID are provided. These validations help ensure that incorrect or invalid data does not enter the system.

The module is implemented using Java and follows Object-Oriented Design principles. The main classes involved are **Cart**, which represents the cart created by a user, and **CartLine**, which represents individual items inside the cart. The **CartService** interface defines the operations that can be performed on the cart, while **DefaultCartService** provides the implementation of these operations using an in-memory data structure to store cart information.

To ensure that the module works correctly, unit tests are written using JUnit to test different cart operations such as creating a cart, validating inputs, and handling edge cases. Maven is used for building and managing the project, and GitHub Actions is used to automatically run tests when code changes are pushed. This helps maintain code quality and ensures that the cart module works properly within the overall food delivery system.

3. Requirements

I. Functional Requirements and Test Cases

FR1 — Create Cart

ID	Description	Expected Result
TC1	Create cart successfully	Cart created with unique cartId and empty item list
TC2	CustomerId is null	System throws IllegalArgumentException
TC3	RestaurantId is null	System throws IllegalArgumentException
TC4	Prevent duplicate cart for same customer and restaurant	Same cart is returned, no new cart created

FR2 — Add Item to Cart

ID	Description	Expected Result
TC5	Add new item to cart	Cart contains ITEM1 x2
TC6	Add multiple items to cart	Cart contains both items

FR3 — Update Item Quantity

ID	Description	Expected Result
TC7	Update existing item quantity	Cart updated to ITEM1 x5

FR4 — Remove Item from Cart

ID	Description	Expected Result
TC8	Remove item when quantity is zero	Item removed from cart

FR5 — Retrieve Cart

ID	Description	Expected Result
TC9	Retrieve cart	Correct cart details returned

FR6 — Clear Cart

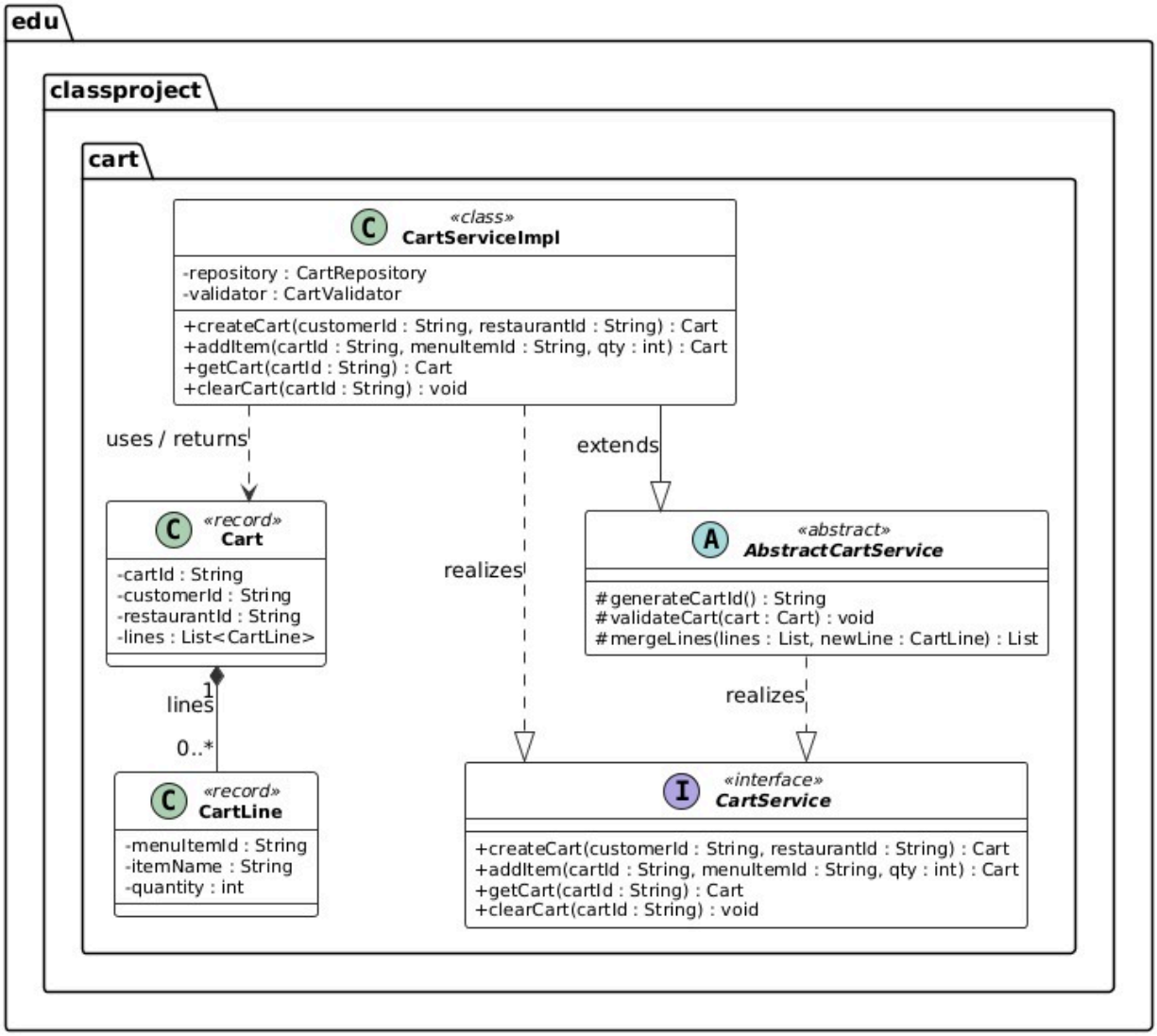
ID	Description	Expected Result
TC11	Clear cart successfully	Cart item list becomes empty

II. Non-Functional Requirements and Test Cases

NFR1 — Input Validation

ID	Description	Expected Result
TC12	Invalid quantity validation	System throws IllegalArgumentException

NFR2 — Data Consistency

ID	Description	Expected Result
TC13	Add item from different restaurant	System rejects operation

4. CLASS UML DIAGRAM